Scientific Python

September 17, 2024

Contents

Introduction

Python is an interpreted language. This makes writing a program easier. However, a consequence is that loops are slow. In scientific computing, it is common to work with large containers and to iterate over them. This requires a specific usage that is different from "general Python", and is made possible thanks to two packages:

- The `numpy` package,
- The `pandas` package.

The `numpy` library provides essential containers in scientific computing, such as vectors or matrices, and supplies a whole set of mathematical functions that make it possible to avoid using loops. The `pandas` package, on the other hand, offers a specific container: the data table, and all the functions to manipulate and study it.

1 Generalities

1.1 Installation

Python is an interpreted language. It requires downloading the interpreter (what is generally called Python), which can be downloaded from the official website: `https://www.python.org/downloads/`.

However, it is not very practical to use it for development as is. It allows executing Python files directly and writing in the console, but this is hardly convenient. For a more user-friendly development, it is useful to use an integrated development environment (IDE). One may for example use RCode: `https://rcode.dev/`.

1.2 First steps

Python has weak typing: one does not specify the type, but all variables have one. This type is implicit.

Assignment of a variable is done with the operator `=`.

```
x = 3
x
```

The function `type` makes it possible to obtain the type of a variable.

```
x = "Python"
type(x)
<class 'str'>
type(True)
<class 'bool'>
type(1)
<class 'int'>
type(1.)
<class 'float'>
```

1.3 Conditionals

The type `bool` represents Booleans: the variable takes the value `True` or `False`.

The operator `and` corresponds to logical AND.

```
True and False
```

The operator `or` corresponds to logical OR.

```
True or False
```

The operator `not` corresponds to logical NOT.

```
not True
```

A condition is written with `if` in the following way:

```
if (condition):
    instructions
```

or without parentheses:

```
if condition:
    instructions
```

The indentation above is fundamental. It defines the scope of the `if`. Indentation is part of the syntax: as long as the next line is indented, it belongs to the `if`.

The condition is a Boolean that takes the value `True` or `False`.

```
x = 7  # This is a comment
if x % 2 == 1:  # If x modulo 2 equals 1
    print("x is odd")  # print writes in the console
```

It is possible to add an alternative using the keyword `else` followed by a colon.

```
x = x + 1  # x = 8
if x % 2 == 1:  # FALSE
    print("x is odd")
else:
    print("x is even")
```

Once again, indentation defines the scope of the `else`.

1.4 Loops

A `for` loop is an instruction that is carried out over a set $i \in I$.

The semantics of the `for` loop is:

```
for i in I:
    instructions
```

The most common case is to perform an instruction over

$$i \in \{0, 1, \ldots, n-1\}$$

for $n \in \mathbb{N}^*$. The function `range` evaluated at n makes it possible to represent this set. The variable I can therefore be `range(n)`.

```
for i in range(4):
    print("Iteration i = " + str(i))  # str(i) converts i to a string
                                      # + concatenates strings
```

It is possible to start the iteration at some $n_0 > 0$ with `range(n0, n)`.

The `while` loop makes it possible to repeat an instruction as long as a condition is satisfied. Its syntax is:

```
while (condition):
    instructions
```

or without parentheses:

```
while condition:
    instructions
```

A `for` loop can always be written as a `while` loop:

```
i = 1
while i <= n:
    instructions
    i = i + 1
```

Be careful with infinite loops when using `while`.

1.5 Functions

A function takes one or several arguments (possibly none) and returns a variable (possibly none).

The syntax to define a function is:

```
def function_name(arg1, ..., argn):
    instructions
```

The call is made with:

```
function_name(arg1, ..., argn)
```

If the function returns a value, it ends with `return`; as soon as a `return` is encountered, the execution of the function stops.

Below is an example with the factorial function.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        f = 1
        for i in range(2, n+1):
            f *= i
        return f

factorial(0), factorial(1), factorial(5)
```

Above, `*=` multiplies the variable on the left of the operator by the element on the right while modifying it; the corresponding line is equivalent to `f = f*i`.

1.6 Containers

1.6.1 Lists

A list is a collection of elements without constraints. It can mix simple variables of any type with objects.

```
l = [1, 4, "RCode"]
print(l)
```

Access to elements is done with `[i]` where i ranges from 0 to the size of the list minus one.

```
[l[0], l[2]]
```

It is possible to include lists inside lists.

```
h = [l, 8]
print(h)
```

Calling the first element of index 0 returns the list `l`.

```
h[0]
```

1.6.2 Tuples

Tuples are exactly like lists except that they are not modifiable.

```
t = (1, 4, "RCode")
print(t)
```

One can even omit the parentheses:

```
t = 1, 4, "RCode"
```

1.6.3 Dictionaries

Dictionaries are lists in which each member is named.

```
c1 = "id"
d = {c1: 1, "x": 4, "label": "RCode"}
print(d)
```

Access is by name:

```
d["label"]
```

2 The numpy package

In scientific computing, it is frequent to have arrays whose elements are all of the same type. The Python list is too flexible.

The `numpy` package introduces a dedicated array object called `numpy.ndarray` (vectors, matrices, etc.).

Always import numpy:

```
import numpy as np
```

2.1 The vector

A vector is a collection of values of the same type.

We can create a vector with `np.array`:

```
x = np.array([1, 3, 7])
x
```

To initialize an array of size n:

```
n = 10
z = np.zeros(n)
z
```

Also `np.ones`, `np.empty`.

`np.arange(a,b)` creates integers on $[a, b[$:

```
np.arange(0, 5)
```

A vector has an attribute `.size`:

```
x.size
```

Indexing:

```
x[0]
```

Boolean mask selection:

```
x = np.array([1, 3, 5])
x[np.array([False, True, True])]
```

Index selection:

```
x[np.array([0, 2, 2, 1])]
```

Slicing:

```
x[0:2]
```

Deleting:

```
np.delete(x, 1)
```

First fundamental rule. All operators applied to two vectors of the same size return a vector where the operator has been applied element by element.

```
y = np.arange(0, 3)
x + y
x * y
x ** y
x == y
```

Second fundamental rule. All operators applied to a vector and a scalar return a vector where the operator has been applied to each element.

```
2*x + 1
x ** 2
x % 2
x <= 2
```

2.2 Aggregation functions

An aggregation function takes a vector and returns a real number.

Sum:

$$\texttt{np.sum(x)} = \sum_{i=1}^{n} x_i.$$

Mean:

$$\texttt{np.mean(x)} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

Variance:

$$\texttt{np.var(x)} = \sum_{i=1}^{n} \left(x_i - \texttt{np.mean}(x)\right)^2.$$

Standard deviation:

$$\texttt{np.std(x)} = \sqrt{\texttt{np.var(x)}}.$$

2.3 Vector language and efficient programming

We say that a function f is *vectorial* if it takes one or several vectors as arguments (and possibly other arguments of size 1) and returns a vector where the function has been applied element by element.

```
for i in range(x.size):
    x[i] = f(i, x[i], z[i])
```

If f is vectorial:

```
x = f(np.arange(x.size), x, z)
```

Remark. Prefer `np.arange(x.size)` over `range(x.size)` when using `numpy`.
Example:

```
np.sum(np.arange(n+1)**2)
```

Norm of $x + y$:

```
np.sqrt(np.sum(x**2 + y**2))
```

Conditional update pattern:

```
ind = h(np.arange(x.size), x, z)
x[ind] = f(np.arange(x.size)[ind], x[ind], z[ind])
```

Example for $y = (x - 1)_+$:

```
y = np.zeros(x.size)
ind = x >= 1
y[ind] = x[ind] - 1
```

2.4 Simulation of random variables

The `numpy` package uses a sequence of pseudo-random numbers generated by an arithmetic recurrence.

- The default algorithm used is the Mersenne–Twister. Its period is $2^{19937} - 1$, which is approximately 10^{6000}.
- This algorithm is often considered to be the best compromise between efficiency and quality.
- The generator is initialized automatically.
- It is possible to choose a seed with `np.random.seed`.
- This allows one to reproduce the same simulations for debugging purposes.

Uniform on $[0, 1]$:

```
np.random.random(size=None)
```

Example:

```
n = 5
U = np.random.random(n)
U
```

Uniform on $[a, b]$:

```
np.random.uniform(low=0., high=1., size=None)
```

Standard normal:

```
np.random.randn(size=None)
```

Example:

```
n = 5
Z = np.random.randn(n)
Z
```

General normal:

```
np.random.normal(loc=0., scale=1., size=None)
```

Some distributions:

- Binomial $B(m, p)$: `np.random.binomial(m, p, n)`
- Poisson $P(\lambda)$: `np.random.poisson(a, n)`
- Geometric starting from 0, $G(p)$: `np.random.geometric(p, n)`
- Negative binomial $NB(m, p)$: `np.random.negative_binomial(m, p, n)`
- Log-normal $LN(m, s^2)$: `np.random.lognormal(m, s, n)`
- Exponential $E(b)$: `np.random.exponential(1/b, n)`
- Gamma $G(a, b)$: `np.random.gamma(a, 1/b, n)`

For densities/cdf/quantiles use `scipy.stats`:

```
import scipy.stats as stats
x = np.linspace(-3., 3., 601)

stats.norm.pdf(x)
stats.norm.cdf(x)
a = np.linspace(0., 1., 21)
stats.norm.ppf(a)
```

2.5 Drawing graphs with matplotlib

A simple and standard way to draw graphs in Python is to use the `matplotlib` package. The usual import is:

```
import matplotlib.pyplot as plt
```

To plot a curve from data points `x` and `y` (same size):

```
x = np.linspace(0., 2*np.pi, 200)
y = np.sin(x)

plt.figure()
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("A␣first␣plot")
plt.grid(True)
plt.show()
```

It is possible to draw several curves on the same figure, and add a legend:

```
y2 = np.cos(x)

plt.figure()
plt.plot(x, y, label="sin")
plt.plot(x, y2, label="cos")
plt.legend()
plt.grid(True)
plt.show()
```

Remark. When studying performance, a log-scale is often useful:

```
plt.figure()
plt.plot(n_values, times_vec, label="vectorized")
plt.plot(n_values, times_loop, label="for-loop")
plt.xscale("log")
plt.yscale("log")
plt.xlabel("n")
plt.ylabel("time␣(s)")
plt.legend()
plt.grid(True)
plt.show()
```

To save the figure to a file, use `plt.savefig("figure.png")` before `plt.show()`.

2.6 Measuring computation time with the time module

Remark (timing functions). To measure computation time in Python, one may use either `time.time()` or `time.perf_counter()`. The function `time.time()` returns the current *wall-clock* time (a Unix timestamp), so it is convenient and often sufficient when the computation is long enough (e.g. a few tenths of a second or more). However, it is not designed for precise benchmarking: its resolution can be limited, and the measurement may be affected by external factors such as system clock adjustments. In contrast, `time.perf_counter()` is a high-resolution *monotonic* timer dedicated to performance measurements, and it typically provides more stable results when the code runs in only a few milliseconds (which is common for vectorized NumPy operations). If you

choose to use `time.time()`, it is recommended to repeat the measurement a few times and keep the minimum (or the median) to reduce measurement noise.

```
import time

t0 = time.perf_counter()
# code to time
t1 = time.perf_counter()
print("Elapsed time:", t1 - t0, "seconds")
```

A convenient pattern is to write a small helper function that measures the runtime of a function call:

```
def time_call(func, *args, repeat=3):
    """Return a robust estimate of the runtime of func(*args)."""
    # warm-up (helps reduce first-call overhead)
    func(*args)

    tmin = float("inf")
    for _ in range(repeat):
        t0 = time.perf_counter()
        func(*args)
        t1 = time.perf_counter()
        tmin = min(tmin, t1 - t0)
    return tmin
```

Finally, to compare a vectorized implementation with a loop implementation, one can measure times for increasing sizes and plot the curves:

```
import numpy as np
import matplotlib.pyplot as plt

n_values = np.array([10**3, 3*10**3, 10**4, 3*10**4, 10**5, 3*10**5, 10**6])

times_vec = []
times_loop = []

for n in n_values:
    times_vec.append(time_call(f_vec, n, repeat=3))
    times_loop.append(time_call(f_loop, n, repeat=3))

times_vec = np.array(times_vec)
times_loop = np.array(times_loop)

plt.figure()
plt.plot(n_values, times_vec, marker='o', label='vectorized')
plt.plot(n_values, times_loop, marker='o', label='for-loop')
plt.xlabel('n')
plt.ylabel('time (s)')
plt.title('Runtime comparison')
plt.grid(True)
plt.legend()
plt.show()
```